API Documentation – Exam-Backend

Movie backend bygget med Javalin, Hibernate/JPA og PostgreSQL. Tabellerne nedenunder dokumentere endpoints som jeg har på nuværende tidspunkt lavet

Basale URL (local development): http://localhost:7070

Deployed URL: https://exam.datudvikling.dk

Beskyttede endpoints kræver headeren Authorization: Bearer <token> (token fås ved login). Rollen står i parentes ved metoden, fx POST (ADMIN) eller POST (USER).

Movies

Method	URL	Request Body (JSON)	Response (JSON)	Error (e)
GET	/api/movies		[movie, movie, ...] (1)	
GET	/api/movies/{id}		movie (1)	(e1), (e2)
POST (ADMIN)	/api/movies	movie request (2)	movie (1), 201	(e3), (e6), (e8)
PUT (ADMIN)	/api/movies/{id}	movie request (2)	movie (1)	(e1), (e2), (e3), (e6), (e8)
DELETE (ADMIN)	/api/movies/{id}		ingen body, 204	(e1), (e2), (e6), (e8)
POST (USER)	/api/movies/{id}/rating	rating request (5)	rating (6)	(e2), (e5), (e6), (e10)

Auth, Genres og Anbefalinger

Method	URL	Request Body (JSON)	Response (JSON)	Error (e)
POST	/api/auth/register	login (3)	user (4), 201	(e4), (e11)
POST	/api/auth/login	login (3)	token (7)	(e7)
GET	/api/genres		[genre, genre, ...] (8)	
GET	/api/genres/{id}		genre (8)	(e2), (e9)
GET (USER)	/api/recommendations		recommendation (9)	(e6)
GET	/api/healthcheck		{ "msg": "API is up and running" }	

Request Body og Response Formats

(1) Movie format

```
{
  "id": Number,
  "title": String,
  "releaseDate": String (date),
```

```
  "tmdbRating": Number,
  "genres": [String]
}
```

Note: genre indeholder kune genre navnet ikke alle genre objekterne

(2) Movie request format

```
{
  "title": String,
  "releaseDate": String (date),
  "tmdbRating": Number,
  "tmdbId": Number
}
```

(3) Login/register format

```
{
  "username": String,
  "password": String
}
```

(4) User format

```
{
  "username": String,
  "roles": [String]
}
```

(5) Rating request format

```
{
  "rating": Number (1-10),
  "comment": String
}
```

(6) Rating format

```
{
  "id": Number,
  "movieId": Number,
  "movieTitle": String,
  "username": String,
  "rating": Number,
  "comment": String
}
```

(7) Token format

```
{
  "token": String,
  "username": String
}
```

(8) Genre format

```
{
```

```
  "id": Number,
  "name": String
}
```

(9) Recommendation format

```
{
  "becauseYouLiked": [String],
  "recommendations": [movie, movie, ...] (1)
}
```

Errors

(e) All errors er reported med den her format (med HTTP-status kode som matcher nummeret)

```
{ "status": statusCode, "msg": "Explains the problem" }
```

- (e1) : { "status": 404, "msg": "Movie with id {id} not found" }
- (e2) : { "status": 400, "msg": "Id must be a number" }
- (e3) : { "status": 400, "msg": "Field 'title' is required" }
- (e4) : { "status": 400, "msg": "Username and password is required" }
- (e5) : { "status": 400, "msg": "Rating must be between 1-10" }
- (e6) : { "status": 401, "msg": "Authorization header is missing" } (eller "Invalid format used" / "Token is not valid or expired")
- (e7) : { "status": 401, "msg": "Invalid credentials" }
- (e8) : { "status": 403, "msg": "User was not authorized with roles: [user]. Needed roles are: [ADMIN]" }
- (e9) : { "status": 404, "msg": "Genre with id {id} not found" }
- (e10) : { "status": 404, "msg": "Movie with id{id}not found" }
- (e11) : { "status": 409, "msg": "Username already taken" }